

Perfect Work Division - Plant Disease Classification Project

Overview

- **Total Team Members:** 5 people
- **Models:** ResNet50, EfficientNetB3, MobileNetV2
- **Dataset:** PlantVillage Dataset
- **Bonus Challenge:** TensorFlow Lite deployment

Person 1: Data Pipeline + Model 1 (ResNet50)

Responsibilities:

A) Data Acquisition & Preprocessing

- Download PlantVillage dataset from official source
- Data cleaning:
 - Remove corrupted images
 - Verify all labels are correct
 - Check class distribution (handle imbalance if needed)
- Image preprocessing:
 - Resize images to consistent size (e.g., 224×224)
 - Normalize pixel values (0-1 or ImageNet normalization)
- Split dataset:
 - Training set (70-80%)
 - Validation set (10-15%)
 - Test set (10-15%)

B) Data Augmentation

- Implement augmentation techniques:
 - Rotation (± 15 -30 degrees)
 - Horizontal/vertical flip
 - Brightness/contrast adjustment

- Zoom/shift
- Color jittering
- **Ablation Study:** Train with and without augmentation to show impact
- Create augmented training dataloaders

C) Model 1 Training: ResNet50

- Load pretrained ResNet50 (ImageNet weights)
- Modify final layer for number of disease classes
- Hyperparameter experimentation:
 - Try different learning rates (1e-3, 1e-4, 1e-5)
 - Test batch sizes (16, 32, 64)
 - Experiment with optimizers (Adam, SGD)
 - Try different freezing strategies
- Train the model:
 - Track training/validation loss and accuracy
 - Implement early stopping
 - Save best model weights
- Document all hyperparameters and training configuration

Person 2: Model 2 (EfficientNetB3) + ALL Metrics & Comparison

Responsibilities:

A) Model 2 Training: EfficientNetB3

- Receive preprocessed data from Person 1
- Load pretrained EfficientNetB3 (ImageNet weights)
- Hyperparameter experimentation:
 - Document all experiments
 - Try different configurations
 - Compare with Person 1's approaches
- Train EfficientNetB3:
 - Track training curves
 - Save best model
 - Document training time and resources

B) Comprehensive Metrics Extraction (ALL 3 MODELS)

This is your main responsibility - you're the **Metrics Coordinator**

For Each Model (ResNet50, EfficientNetB3, MobileNetV2):

1. Basic Metrics:

- Overall Accuracy
- Precision (macro, micro, weighted, per-class)
- Recall (macro, micro, weighted, per-class)
- F1-Score (macro, micro, weighted, per-class)

2. Confusion Matrix:

- Generate full confusion matrix
- Create heatmap visualization
- One for each model

3. Per-Disease-Type Performance:

- Detailed breakdown for each plant disease class
- Identify which diseases are hardest to classify
- Create performance comparison table by disease type

4. Classification Report:

- Complete sklearn classification report
- Per-class precision, recall, F1, support
- Save as formatted table

5. Training Curves:

- Loss curves (train vs validation)
- Accuracy curves (train vs validation)
- For all three models

6. ROC Curves (Optional):

- Multi-class ROC if time permits
- AUC scores

C) Models Comparison Analysis

- Create comprehensive comparison table:

Metric	ResNet50	EfficientNetB3	MobileNetV2
Accuracy	X%	Y%	Z%
Precision	...	...	...
Recall	...	...	...
F1-Score	...	...	...
Training Time	...	...	...

Model Size	...		...		...	
Inference Speed	...		...		...	

- Create visualization comparing all models:
 - Bar chart comparing accuracy/F1
 - Side-by-side confusion matrices
 - Performance by disease type comparison
- Write analysis:
 - Which model performs best overall?
 - Which model is best for specific disease types?
 - Trade-offs (accuracy vs speed vs size)

Person 3: Model 3 (MobileNetV2) + Grad-CAM

Responsibilities:

A) Model 3 Training: MobileNetV2

- Receive preprocessed data from Person 1
- Load pretrained MobileNetV2 (ImageNet weights)
- Hyperparameter experimentation:
 - Document experiments
 - Focus on lightweight/efficient training
- Train MobileNetV2:
 - Track training curves
 - Save best model
 - Measure inference speed (important for mobile)

B) Grad-CAM Visualization (Explainability)

Generate Grad-CAM heatmaps for **all 3 models** (coordinate with Person 1 and 2):

For Each Model:

- 1. Correct Predictions:**
 - a. Select 5 correctly classified images
 - b. Generate Grad-CAM heatmaps
 - c. Show what the model is "looking at"
- 2. Incorrect Predictions:**
 - a. Select 5 incorrectly classified images

- b. Generate Grad-CAM heatmaps
- c. Analyze why the model failed

3. Sample Diversity:

- a. Cover different disease types
- b. Show variety of model attention patterns

Total: 30 images (5 correct + 5 incorrect × 3 models)

- Save all heatmap images organized by model
- Create comparison showing how different architectures focus on different features

Person 4: GUI Development (Streamlit) + Integration

Responsibilities:

A) Streamlit GUI - Main Interface

Page 1: Model Prediction Interface

- **Image Input:**
 - Option 1: Upload image (JPG, PNG)
 - Option 2: Use webcam capture (real-time bonus)
 - Display uploaded/captured image
- **Model Selection:**
 - Dropdown to select: ResNet50 / EfficientNetB3 / MobileNetV2
 - Show model info (size, speed, accuracy)
- **Prediction Display:**
 - Top-3 predicted classes with confidence scores
 - Display as progress bars or formatted table
 - Show actual disease name (not just class number)
- **Grad-CAM Visualization:**
 - Display Grad-CAM heatmap overlay
 - Show original image + heatmap side-by-side
 - Explain what model is focusing on

Page 2: Model Performance Dashboard

- Display metrics from Person 2:
 - Overall accuracy comparison (bar chart)
 - Confusion matrices for each model

- Per-disease performance table
- Training curves

Page 3: About/Documentation

- Project description
- Dataset information
- Team members and roles
- How to use the application
- Model architecture descriptions

B) Backend Integration

- Load all 3 trained models efficiently
- Implement preprocessing pipeline for uploaded images
- Connect Grad-CAM functionality
- Handle errors gracefully (invalid images, loading failures)
- Optimize for reasonable inference speed

C) Deployment Preparation

- Create `requirements.txt` with all dependencies
- Test application locally on multiple machines
- Ensure cross-platform compatibility
- Create `app.py` - main Streamlit application
- Write deployment instructions (how to run locally)



Person 5: GitHub Management + Documentation + Final Report



Responsibilities:

A) GitHub Repository Management

Repository Structure:

```
plant-disease-classification/  
├── README.md  
├── requirements.txt  
└── .gitignore
```

```
├── data/
│   ├── raw/
│   ├── processed/
│   └── README.md
├── notebooks/
│   ├── person1_data_and_model1.ipynb
│   ├── person2_model2_and_metrics.ipynb
│   ├── person3_model3_gradcam_tflite.ipynb
│   └── person4_gui_development.ipynb
├── src/
│   ├── data_preprocessing.py
│   ├── model_training.py
│   ├── evaluation.py
│   └── utils.py
├── models/
│   ├── resnet50_best.pth
│   ├── efficientnetb3_best.pth
│   ├── mobilenetv2_best.pth
│   ├── mobilenetv2.tflite
│   └── model_configs/
├── results/
│   ├── metrics/
│   ├── confusion_matrices/
│   ├── gradcam/
│   └── comparison_tables/
├── gui/
│   ├── app.py
│   ├── requirements.txt
│   └── assets/
├── docs/
│   ├── FINAL_REPORT.pdf
│   ├── TEAM_ROLES.md
│   └── SETUP_INSTRUCTIONS.md
└── presentation/
    └── slides.pptx
```

Git Management:

- Create repository with proper .gitignore
- Set up branch protection rules
- Create branches for each person (optional)

- Monitor commits from all team members
- Ensure everyone contributes with individual commits
- Review and merge pull requests
- Maintain clean commit history with meaningful messages

B) Comprehensive README.md

Must include:

1. Project Header:

- a. Project title and banner image
- b. Team name and members
- c. University/course information

2. Project Overview:

- a. Problem statement
- b. Business value/applications
- c. Dataset description and source
- d. Key features

3. Dataset Information:

- a. PlantVillage dataset details
- b. Number of classes (disease types)
- c. Number of images
- d. Class distribution
- e. Sample images

4. Models Used:

- a. ResNet50 architecture overview
- b. EfficientNetB3 architecture overview
- c. MobileNetV2 architecture overview
- d. Why these models were chosen
- e. Transfer learning approach

5. Methodology:

- a. Data preprocessing steps
- b. Augmentation techniques
- c. Training strategy
- d. Hyperparameter tuning approach
- e. Evaluation metrics

6. Results:

- a. Performance comparison table
- b. Best performing model
- c. Confusion matrices
- d. Key findings

- e. Embed graphs and visualizations
- 7. Explainability:**
 - a. Grad-CAM approach
 - b. Sample visualizations
 - c. Interpretation
- 8. Bonus Feature:**
 - a. TFLite deployment
 - b. Performance comparison
 - c. Mobile deployment possibilities
- 9. GUI Application:**
 - a. Features overview
 - b. Screenshots
 - c. How to run
- 10. Setup Instructions:**
 - a. Prerequisites
 - b. Installation steps
 - c. How to run training scripts
 - d. How to run GUI
 - e. Troubleshooting
- 11. Team Roles:**
 - a. Each member's responsibilities
 - b. Contribution breakdown
- 12. References:**
 - a. Dataset citation
 - b. Papers/resources used
 - c. Libraries and frameworks

C) Final Report (PDF)

Structure:

1. Abstract (1 page)

- Project summary
- Problem statement
- Approach
- Key results
- Conclusion

2. Introduction (1-2 pages)

- Background on plant diseases

- Importance of early detection
- Deep learning in agriculture
- Project objectives
- Dataset description

3. Related Work (1 page)

- Existing approaches to plant disease classification
- Comparison with our approach

4. Methodology (3-4 pages)

- Data collection and preprocessing
- Augmentation techniques and ablation study
- Model architectures (ResNet50, EfficientNetB3, MobileNetV2)
- Transfer learning approach
- Training procedure
- Hyperparameter tuning
- Evaluation metrics

5. Experiments and Results (3-4 pages)

- Experimental setup
- Training details for each model
- Performance comparison table
- Confusion matrices
- Per-disease-type analysis
- Training curves
- Best performing model analysis
- Grad-CAM visualizations and interpretation

6. Bonus Feature: TFLite Deployment (1 page)

- Conversion process
- Performance comparison
- Mobile deployment potential

7. GUI Application (1 page)

- Features description
- Screenshots
- User workflow

- Integration approach

8. Discussion (1-2 pages)

- Analysis of results
- Why certain models performed better
- Challenges faced
- Limitations
- Comparison with state-of-the-art

9. Conclusion and Future Work (1 page)

- Summary of achievements
- Key takeaways
- Potential improvements
- Real-world applications
- Future research directions

10. Team Contributions (1 page)

- Detailed breakdown of each member's work
- Collaboration process

11. References

- All citations properly formatted

Appendices:

- Additional visualizations
- Code snippets (if relevant)
- Hyperparameter tables

D) Additional Documentation

- TEAM_ROLES.md - detailed role descriptions
- SETUP_INSTRUCTIONS.md - comprehensive setup guide
- CONTRIBUTING.md - how to contribute to the project
- LICENSE - project license
- CHANGELOG.md - version history

E) Quality Assurance

- Review all notebooks for completeness
- Ensure all code runs without errors
- Verify all links work
- Check that all deliverables are in repository
- Proofread all documentation
- Ensure consistent formatting
- Test installation instructions
- Validate that requirements.txt is complete

Grading Breakdown Coverage

Component	Points	Responsibility	Status
Dataset preparation	Part of 5	Person 1	✓
Preprocessing	Part of 5	Person 1	✓
Model design	Part of 5	Persons 1,2,3	✓
Training	Part of 5	Persons 1,2,3	✓
Hyperparameter tuning	Part of 5	Persons 1,2,3	✓
Accuracy metrics	3 pts	Person 2	✓
Precision/Recall	3 pts	Person 2	✓
Confusion Matrix	3 pts	Person 2	✓
3 Architecture Comparison	3 pts	Person 2	✓
Grad-CAM (Explainability)	1 pt	Person 3	✓
GUI Implementation	1 pt	Person 4	✓
Image upload/webcam	Part of 1	Person 4	✓
Top-3 predictions	Part of 1	Person 4	✓
Confidence scores	Part of 1	Person 4	✓
GitHub Repository	2 pts	Person 5	✓
Commits from all members	Part of 2	Everyone	✓
README documentation	Part of 2	Person 5	✓
Clear structure	Part of 2	Person 5	✓
Per-disease evaluation	Required	Person 2	✓
Color/orientation augmentation	Required	Person 1	✓
BONUS: TFLite deployment	+5 pts	unassigned	✓
Total Base Points	20 pts	—	✓

Total with Bonus

25 pts —



GitHub Workflow:

- Each person commits their work regularly (minimum 5-10 commits)
- Use meaningful commit messages
- Tag Person 5 for integration issues
- Use issues/project boards for task tracking

Quality Checklist (All Members)

Before final submission, verify:

- ☐ All 3 models trained successfully
- ☐ All model weights saved and accessible
- ☐ Comprehensive metrics for all models
- ☐ Confusion matrices for all models
- ☐ Grad-CAM for all models (30 images total)
- ☐ GUI runs without errors
- ☐ GUI displays all required features
- ☐ TFLite model converted and tested
- ☐ All notebooks run from start to finish
- ☐ README is comprehensive and well-formatted
- ☐ Final report is complete (15-20 pages)
- ☐ All team members have commits
- ☐ Repository is well-organized
- ☐ requirements.txt includes all dependencies
- ☐ Setup instructions work on fresh environment
- ☐ All images/visualizations are high quality
- ☐ No placeholder text ("TODO", "Coming soon")
- ☐ Presentation slides prepared

Success Criteria

Your project will excel if:

1. ✓ All 3 models achieve >85% accuracy
2. ✓ Clear performance comparison with insights
3. ✓ Grad-CAM visualizations are meaningful
4. ✓ GUI is intuitive and functional
5. ✓ TFLite bonus is implemented (+5 pts)
6. ✓ Documentation is professional and complete
7. ✓ GitHub shows collaborative work
8. ✓ Per-disease analysis identifies patterns
9. ✓ Augmentation ablation shows clear benefit
10. ✓ Final report is publication-quality

This division ensures:

- ✓ **No overlap** in work
- ✓ **All requirements covered**
- ✓ **Balanced workload**
- ✓ **Clear deliverables** for each person
- ✓ **Efficient coordination**
- ✓ **Bonus points** included
- ✓ **Full 20 + 5 points** achievable

Bonus: TensorFlow Lite Deployment (+5 pts)

- Convert MobileNetV2 to TFLite format:
 - Optimize for mobile inference
 - Apply quantization if possible
 - Test converted model accuracy
- Create comparison:
 - Original model size vs TFLite size
 - Inference speed comparison
 - Accuracy before/after conversion
- Document deployment process:
 - Conversion steps
 - How to use TFLite model
 - Performance benchmarks